МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт кибербезопасности и цифровых технологий (ИКБ)
Базовая кафедра «Прикладное программирование»

ОТЧЁТ ПО ПРОИЗВОДСТВЕННОЙ ПРАКТИКЕ
Проектно-технологическая практика

Тема практики: «Проектирование и выбор типа рекомендательной системы»
приказ Университета о направлении на практику от «29» августа 2025 г. № 7118-С

Отчет представлен к рассмотрению: Студент группы БФБО-03-22	«20» 12	2025 г.	_____ (подпись)	А.А. Артемов
Отчет утвержден. Допущен к защите:				
Руководитель практики от кафедры	«20» 12	2025 г.	_____ (подпись)	С.П. Луцкан

Москва 2025 г.

МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт кибербезопасности и цифровых технологий (ИКБ)
Базовая кафедра «Прикладное программирование»

ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ НА ПРОИЗВОДСТВЕННУЮ ПРАКТИКУ
Проектно-технологическая практика

Студенту 4 курса учебной группы БФБО-03-22
Артемова Андрея Александровича

Место и время практики: РТУ МИРЭА, базовая кафедра «Прикладное программирование», 01 сентября 2025 г. по 20 декабря 2025 г.

Должность на практике: студент

1. ЦЕЛЕВАЯ УСТАНОВКА: развитие способностей в области проектной деятельности путем применения современных методов и инструментальных средств прикладной информатики

2. СОДЕРЖАНИЕ ПРАКТИКИ:

2.1. Изучить: информационные и прикладные процессы работы информационной системы

2.2. Практически выполнить: применить современные инструментальные средства для оптимизации, улучшения и производительности информационной системы

2.3. Ознакомиться: с уровнем развития информационных и прикладных процессов информационной системы

3. ДОПОЛНИТЕЛЬНОЕ ЗАДАНИЕ: оформить презентационный материал

4. ОРГАНИЗАЦИОННО-МЕТОДИЧЕСКИЕ УКАЗАНИЯ: в процессе практики рекомендуется использовать методические рекомендации по прохождению проектно-технологической практики

Заведующий кафедрой:

«01» сентября 2025 г. _____ (подпись) С.П. Луцкан

СОГЛАСОВАНО:
Руководитель практики от кафедры
«01» сентября 2025 г. _____ (подпись) С.П. Луцкан

Задание получил
«01» сентября 2025 г. _____ (подпись) А.А. Артемов

МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»

РТУ МИРЭА

РАБОЧИЙ ГРАФИК ПРОВЕДЕНИЯ
Проектно-технологическая практика

Студента А.А. Артемова 4 курса группы БФБО-03-22 очной формы обучения, обучающегося по направлению подготовки 02.03.02 «Фундаментальная информатика и информационные технологии», профиль «Искусственный интеллект и машинное обучение».

Неделя	Сроки выполнения	Этап	Отметка о выполнении
1	01.09.2025	Подготовительный этап, включающий в себя организационное собрание	Выполнено
2	03.10.2025	Выполнение задания по практике в соответствии с выданным заданием студента	Выполнено
3	21.11.2025	Подготовка отчета и презентационного материала по практике	Выполнено
4	20.12.2025	Защита отчета по производственной практике у руководителя практики	Выполнено

Содержание практики и планируемые результаты согласованы с руководителем практики от профильной организации

Согласовано:

Заведующий кафедрой _____/С.П. Луцкан /

Руководитель практики от кафедры _____/ С.П. Луцкан /

Обучающийся _____/ А.А. Артемов/

ОТЧЁТ
Проектно-технологическая практика

студента 4 курса учебной группы БФБО-03-22
Института кибербезопасности и цифровых технологий Университета РТУ МИРЭА
Андрея Александровича Артемова

1. Практику проходил с 01.09.2025 г. по 20.12.2025 г. в РТУ МИРЭА, базовая кафедра «Прикладное программирование», студент
(место прохождения практики и должность)

2. Задание на практику выполнил в полном объеме
(указать: в полном объеме или частично)

Не выполнены следующие задания:

(указать также причины невыполнения)

Подробное содержание выполненной на практике работы и достигнутые результаты: изучена структура организации и рабочего процесса, оптимизирован механизм передачи данных между компонентами системы, оптимизированы процессы синхронизации данных, сэкономлены серверные ресурсы организации, разработан интерфейс получения данных. Изучены подходы к реализации архитектуры современных приложений, исследованы компоненты современных приложений

Предложения по совершенствованию организации и прохождения практики:
предложений нет

Студент _____ (А.А. Артемов)
(подпись)

«20» декабря 2025 г.

Заключение руководителя практики

Приобрел следующие профессиональные навыки: практикант продемонстрировал профессиональные умения и навыки, знание и понимание прикладной области, задач, требующих решения в прикладной области, современные подходы и средства решения прикладных задач разных классов, умение находить и работать с различными источниками информации по профессиональной деятельности, структурировать отчет с учетом тематики исследования

Проявил себя как: дисциплинированный ответственный специалист: соблюдал сроки календарного графика практики, регулярно отчитывался о проделанных этапах работ; за срок прохождения практики не получил ни одного замечания - проявляет инициативу, четко и в определенные сроки выполняет задания; в любой ситуации уважителен в общении с другими.

«20» декабря 2025 г.

Отчет проверил:
Руководитель практики от Университета

_____ (С.П. Луцкан)
(подпись)

Проведенные инструктажи:

Охрана труда: «01» сентября 2025 г.

Инструктирующий _____ С.П. Луцкан
(подпись)

Инструктируемый _____ А.А. Артемов
(подпись)

Техника безопасности: «01» сентября 2025 г.

Инструктирующий _____ С.П. Луцкан
(подпись)

Инструктируемый _____ А.А. Артемов
(подпись)

Пожарная безопасность: «01» сентября 2025 г.

Инструктирующий _____ С.П. Луцкан
(подпись)

Инструктируемый _____ А.А. Артемов
(подпись)

Правила внутреннего трудового распорядка: «01» сентября 2025 г.

Инструктирующий _____ С.П. Луцкан
(подпись)

Инструктируемый _____ А.А. Артемов
(подпись)

С правилами внутреннего распорядка ознакомлен: «01» сентября 2025 г.

_____ А.А. Артемов
(подпись)

Содержание

- Введение 7
- 1. Популярные алгоритмы в рекомендательных системах 9
- 2. Контентная Рекомендательная система 11
- 3. Выбор технологического стэка для всего приложения 15
  - 3.1. Технологический стэк для рекомендательной системы 15
  - 3.2. Технологический стэк для серверной части 15
  - 3.3. Технологический стэк для клиентской части 16
- 4. Проектирование всего приложения 18
- Заключение 20
- Список используемой литературы 22

Введение

В современном мире количество фильмов и сериалов на стриминговых платформах и в онлайн— кинотеатрах исчисляется сотнями тысяч. Пользователь, заходя на сервис, сталкивается с парадоксом выбора: найти что— то действительно интересное среди такого объёма контента практически невозможно без помощи интеллектуальных инструментов. Именно поэтому рекомендательные системы стали ключевым элементом успеха Netflix, YouTube, Кинопоиска HD, IVI и многих других платформ: по данным исследований, более 75–80 % просмотров начинается именно с рекомендаций.

Одним из самых простых, понятных и при этом удивительно эффективных подходов к рекомендациям фильмов является контентная фильтрация на основе схожести описаний и метаданных (content— based filtering). В отличие от коллаборативных методов, этот подход не требует истории оценок миллионов пользователей и полностью решает проблему холодного старта для новых пользователей: рекомендации строятся исключительно на основе характеристик самих фильмов — сюжета, жанров, ключевых слов, актёров и режиссёров.

Цель данной курсовой работы — разработать и исследовать контентную рекомендательную систему для фильмов, использующую векторизацию текстовых признаков (обзор сюжета + жанры) и косинусное сходство в пространстве признаков, а также показать, насколько эффективным может быть этот классический подход при правильной предобработке данных и выборе параметров.

Для достижения поставленной цели решаются следующие задачи:

Изучить теоретические основы контентной фильтрации и её преимущества перед другими типами рекомендательных систем в задаче рекомендаций фильмов.

Подготовить датасет с метаданными фильмов, выполнить очистку и объединение текстовых полей в единый тег.

Провести качественный анализ результатов: проверить рекомендации для популярных фильмов, выявить сильные и слабые стороны подхода.

Практическая значимость работы заключается в создании полностью рабочей, легко интерпретируемой и быстро запускаемой рекомендательной системы, которая:

не требует больших вычислительных ресурсов,

работает даже на небольшом датасете,

даёт интуитивно понятные и объяснимые рекомендации,

может быть легко интегрирована в веб— приложение или мобильный сервис.

Таким образом, работа демонстрирует, что даже «простой» подход на основе контентной фильтрации и косинусного сходства текстовых описаний способен давать качественные и логически обоснованные рекомендации фильмов, оставаясь при этом одним из лучших выборов для небольших проектов, прототипов, а также в случаях, когда важны объяснимость и независимость от пользовательских оценок.

1. Популярные алгоритмы в рекомендательных системах

Современные рекомендательные системы используют несколько основных подходов, каждый из которых имеет свои сильные и слабые стороны. Ниже рассмотрены наиболее распространённые алгоритмы.

Контентные рекомендательные системы (Content— based filtering) Основаны на анализе характеристик (содержимого) объектов. Для каждого элемента (фильм, товар, статья) создаётся вектор признаков (например, жанр, ключевые слова, теги, описание). Профиль пользователя формируется на основе объектов, которые ему понравились. Рекомендации выдаются путём поиска объектов, наиболее похожих на уже понравившиеся. Для представления текстов чаще всего применяются методы Bag— of— Words, TF— IDF, а в современных реализациях — эмбеддинги (Word2Vec, BERT, Sentence— BERT). Сходство между объектами и профилем пользователя обычно измеряется с помощью косинусного расстояния. Преимущества: простота реализации, отсутствие проблемы «холодного старта» для новых объектов, рекомендации легко объяснимы. Недостатки: ограниченность «пузыря фильтров» (рекомендуется только похожее), низкая серендипность.

Коллаборативная фильтрация (Collaborative filtering) Делится на два основных подвида:

User— based CF: «Похожие на тебя пользователи любят вот это». Находятся пользователи с похожими вкусами и рекомендуются их предпочтения.

Item— based CF: «Те, кто купил/посмотрел этот товар, также покупали вот этот». Чаще используется на практике (например, в Amazon), потому что количество товаров обычно меньше количества пользователей, и сходство между товарами меняется реже. Сходство вычисляется через косинусное расстояние или корреляцию Пирсона на основе матрицы «пользователь–объект». Преимущества:

высокая точность при большом количестве данных, способность находить неожиданные рекомендации. Недостатки: проблемы холодного старта для новых пользователей и объектов, разреженность данных.

Матричная факторизация (Matrix Factorization) Самый популярный и эффективный на сегодняшний день подход (SVD, SVD++, ALS, Funk— SVD, Neural CF). Матрица взаимодействий R (пользователи × объекты) раскладывается на два низкоранговых произведения: $R \approx P \times Q^T$, где P — скрытые факторы пользователей, Q — скрытые факторы объектов (обычно размерность 20–200). Именно этот подход принёс победу команде BellKor в конкурсе Netflix Prize в 2009 году. Преимущества: высокая точность, масштабируемость (особенно ALS в Spark), возможность работы с неявными отзывами. Недостатки: сложнее интерпретировать рекомендации.

Гибридные системы В реальных сервисах (Netflix, YouTube, Spotify, Amazon) почти всегда используются комбинации подходов:

контентные + коллаборативные;

матричная факторизация + контентные признаки;

двухэтапные модели (кандидаты → ранжирование): сначала грубый отбор сотен кандидатов (например, через Item— based CF или LightFM), затем точное ранжирование с помощью градиентного бустинга (XGBoost, LightGBM) или нейронных сетей. Именно гибридные подходы дают наибольшую точность в промышленных системах.

Нейронные и последовательные модели (современные тенденции)

Рекуррентные сети (GRU4Rec, NARM) и трансформеры (SASRec, BERT4Rec) — для последовательных рекомендаций.

Двухбашенные нейронные сети (Two— Tower) — в YouTube, AliExpress, Avito.

Графовые нейронные сети (PinSage, LightGCN) — когда важны связи между объектами и пользователями.

2. Контентная Рекомендательная система

В данной работе в качестве основного алгоритма рекомендательной системы выбран **контентный подход (content— based filtering)** с использованием векторизации текста методом Bag— of— Words и косинусного сходства в качестве метрики близости. Данный выбор обусловлен следующими причинами:

высокая интерпретируемость получаемых рекомендаций (пользователю легко объяснить, почему ему рекомендован именно этот фильм);

отсутствие проблемы «холодного старта» для новых объектов (новый фильм сразу может быть рекомендован, если у него есть описание и жанры);

простота реализации и отладки;

хорошее качество рекомендаций при наличии качественных текстовых метаданных (overview и жанры);

высокая скорость работы на предвычисленной матрице сходства;

данный подход широко применяется в промышленности как базовый компонент гибридных систем (Netflix, Spotify, Bookmate и др.).

Рассмотрим абстрактный пример на фильмах, чтобы понять как работает данный алогритм

Таблица 1 — Данные для примера

Название фильма	**Краткое описание**
Космическая одиссея	космос корабль планеты будущее роботы фантастика приключения
Звёздные войны	космос джедаи ситхи световые мечи империя фантастика приключения
Матрица	виртуальная реальность хакер избранный фантастика боевик
Властелин колец	эльфы орки кольцо средиземье магия фэнтези приключения
Гарри Поттер	волшебная школа магия друзья драконы фэнтези приключения
Терминатор	робот из будущего киборг оружие боевик фантастика

Преобразование текста в векторы (Bag— of— Words)

Из всех тегов собирается общий словарь. Берутся только самые понятные и важные слова (для простоты):

Словарь: космос, корабль, планеты, роботы, джедаи, мечи, империя, реальность, хакер, эльфы, орки, кольцо, магия, волшебная, школа, драконы, терминатор, киборг, оружие, фантастика, приключения, боевик, фэнтези

Каждый фильм → вектор (здесь помечены только ключевые слова, остальные = 0):

Таблица 2— Векторное представление фильмов

Фильм	**космос**	**джедаи**	**магия**	**эльфы**	**киборг**	**фантастика**	**приключения**
Космическая одиссея	1	0	0	0	0	1	1
Звёздные войны	1	1	0	0	0	1	1
Матрица	0	0	0	0	0	1	0
Властелин колец	0	0	1	1	0	0	1
Гарри Поттер	0	0	1	0	0	0	1
Терминатор	0	0	0	0	1	1	0

Далее необходимо посчитать косинусное сходство с помошью данной формулы.

$$cosineSimilarity(A, B) = \frac{\mathrm{A} \times B}{\|A\| \times \|B\|}, \quad (2.1)$$

где $\mathrm{A} \times B$— скалярное произведение векторов (сумма произведений соответствующих элементов)

$\|A\|$— длина (норма) вектора A

$\|B\|$— длина (норма) вектора B

Значение всегда от 0 до 1 (чем ближе к 1 — тем похожи).

Перейдём к расчёту косинусового сходства “Звёздные войны” со всеми фильмами.

1. Звёздные войны ↔ Звёздные войны

   Скалярное произведение $= 1 + 1 + 0 + 0 + 0 + 1 + 1 = 4$

   Длина вектора $= \sqrt{4} = 2$

   Косинус $= 4 \div (2 \times 2) = 1.00$

2. Звёздные войны ↔ Космическая одиссея

   Скалярное произведение:$= 1 \times 1 + 1 \times 0 + 0 \times 0 + 0 \times 0 + 0 \times 0 + 1 \times 1 + 1 \times 1 = 3$

   Длина «Звёздные войны» $= 2$

   Длина «Космическая одиссея» $= s\sqrt{1 + 0 + 0 + 0 + 0 + 1 + 1} = 1.732$

   Косинус = $3 \div (2 \times 1.732) \approx 0.866$

3. Звёздные войны ↔ Матрица

   Скалярное произведение: $1 \times 0 + 1 \times 0 + 0 \times 0 + 0 \times 0 + 0 \times 0 + 1 \times 1 + 1 \times 0 = 1$

   Длина «Матрица» $= \sqrt{0 + 0 + 0 + 0 + 0 + 1 + 0} = 1$

   Косинус $= 1 \div (2 \times 1) = 0.50$

4. Звёздные войны ↔ Властелин колец

   Скалярное произведение: $1 \times 0 + 1 \times 0 + 0 \times 1 + 0 \times 1 + 0 \times 0 + 1 \times 0 + 1 \times 1 = 1$

   Длина «Властелин колец» $= \sqrt{0 + 0 + 1 + 1 + 0 + 0 + 1} = \sqrt{3} \approx 1.732$

   Косинус $= 1 \div (2 \times 1.732) \approx 0.289$

5. Звёздные войны ↔ Гарри Поттер

   Скалярное произведение: только «приключения» →1

   Длина «Гарри Поттер»= $\sqrt{2} \approx 1.414$ =

   Косинус $= 1 \div (2 \times 1.414 \approx 0.354$

6. Звёздные войны ↔ Терминатор

   Скалярное произведение: только «фантастика» →1

   Длина «Терминатор» $= \sqrt{0+0+0+0+1+1+0} = \sqrt{2} \approx 1.414$

   Косинус$= 1 \div (2 \times 1.414) \approx 0.354$ =

Таблица 3 — Итоговая таблица сходства

Название фильма	**Косинусное сходство**	**Будет ли в топ— 5 рекомендаций?**
Звёздные войны	1.000	(сам фильм — пропускаем)
Космическая одиссея	0.866	Да — 1 место
Матрица	0.500	Да — 2 место
Гарри Поттер	0.354	Да — 3 место
Терминатор	0.354	Да — 4 место
Властелин колец	0.289	Нет — слишком низко

При запросе рекомендаций к фильму «Звёздные войны» алгоритм выдаст в первую очередь те фильмы, у которых больше всего общих значимых слов в описании и жанрах. Фильмы про магию и фэнтези («Властелин колец») получат низкий балл и не попадут в рекомендации — что полностью соответствует логике контентного подхода.

3. Выбор технологического стэка для всего приложения

3.1. Технологический стэк для рекомендательной системы

Для реализации рекомендательной системы фильмов был выбран следующий технологический стек:

Таблица 4 — Технологический стэк для рекомендательной системы

Компонент	Технология / библиотека	Обоснование выбора
Язык программирования	Python [1]	Де— факто стандарт в области анализа данных и машинного обучения. Огромное сообщество, множество готовых библиотек.
Работа с данными	Pandas [2]	Самая популярная библиотека для обработки табличных данных. Удобные операции фильтрации, объединения, очистки данных.
Векторизация текста	CountVectorizer (sklearn.feature_extraction.text)	Классическая и проверенная временем реализация метода Bag— of— Words. Простота, высокая скорость, встроенная обработка стоп— слов и ограничение словаря.
Вычисление сходства	cosine_similarity (sklearn.metrics.pairwise)	Оптимизированная на C— подобных уровнях реализация косинусного сходства. Работает с разреженными матрицами, крайне быстро считает миллионы пар.
Сериализация модели	Pickle [3]	Стандартный способ сохранения предобученных объектов Python [1] (матрицы сходства, векторайзера). Быстро, просто, не требует внешних зависимостей.
Среды разработки и выполнения	Jupyter Notebook / Google Colab / VS Code	Интерактивная разработка, удобная визуализация промежуточных результатов, лёгкое прототипирование.

3.2. Технологический стэк для серверной части

Для предоставления рекомендаций пользователям в режиме реального времени будет реализована серверная часть в виде REST API. Выбор технологий для серверной части обоснован следующими требованиями: высокая скорость ответа, простота разработки и развёртывания, поддержка асинхронности, минимальные накладные расходы.

Таблица 5 — Технологический стэк серверной части

Компонент	Технология / библиотека	Обоснование выбора и подробное описание
Веб— фреймворк	FastAPI [4]	Один из самых быстрых Python— фреймворков (на базе Starlette + Pydantic). Автоматическая генерация OpenAPI/Swagger, валидация, асинхронность.
Язык программирования	Python [1]	Единый язык проекта, огромная экосистема, простота поддержки.
Сериализация модели	Pickle [3]	Быстрое сохранение и загрузка предвычисленной матрицы сходства и датафрейма в бинарном виде.
Работа с данными	Pandas [2]	Мгновенный доступ по индексам к метаданным фильмов (title → index → строка в матрице).
HTTP— запросы к внешним API	requests [5]	Надёжная синхронная библиотека для обращения к публичным API.
Получение постеров и метаданных	The Movie Database API (TMDb) [6]	Официальный и самый полный открытый источник актуальных постеров, описаний и метаданных фильмов.

3.3. Технологический стэк для клиентской части

Клиентская часть рекомендательной системы будет реализована как современное одностраничное приложение (SPA).

Таблица 6 — Технологический стэк Клиентской части

Компонент	Технология / библиотека	Обоснование выбора и подробное описание
Фреймворк	React [7]	Де— факто стандарт фронтенд— разработки. Компонентный подход, высокая производительность, огромная экосистема и сообщество.
Язык программирования	TypeScript [8]	Статическая типизация значительно снижает количество багов, упрощает рефакторинг и поддержку кода, особенно при работе с API— ответами.
UI— библиотека и компоненты	shadcn/ui + Tailwind CSS [9]	Современный «головной» UI— кит. Не является готовой библиотекой — это набор копируемых, полностью кастомизируемых компонентов (Button, Card, Input, Skeleton и т.д.). Полный контроль над стилями.
Бандлер и среда разработки	Vite [10]	Самый быстрый на сегодняшний день инструмент сборки.
HTTP— клиент	Axios [11]	Простая и надёжная работа с REST API бэкенда.

4. Проектирование всего приложения

Весь проект представляет собой классическую клиент— серверную систему рекомендаций фильмов по содержанию и состоит из четырёх основных блоков:

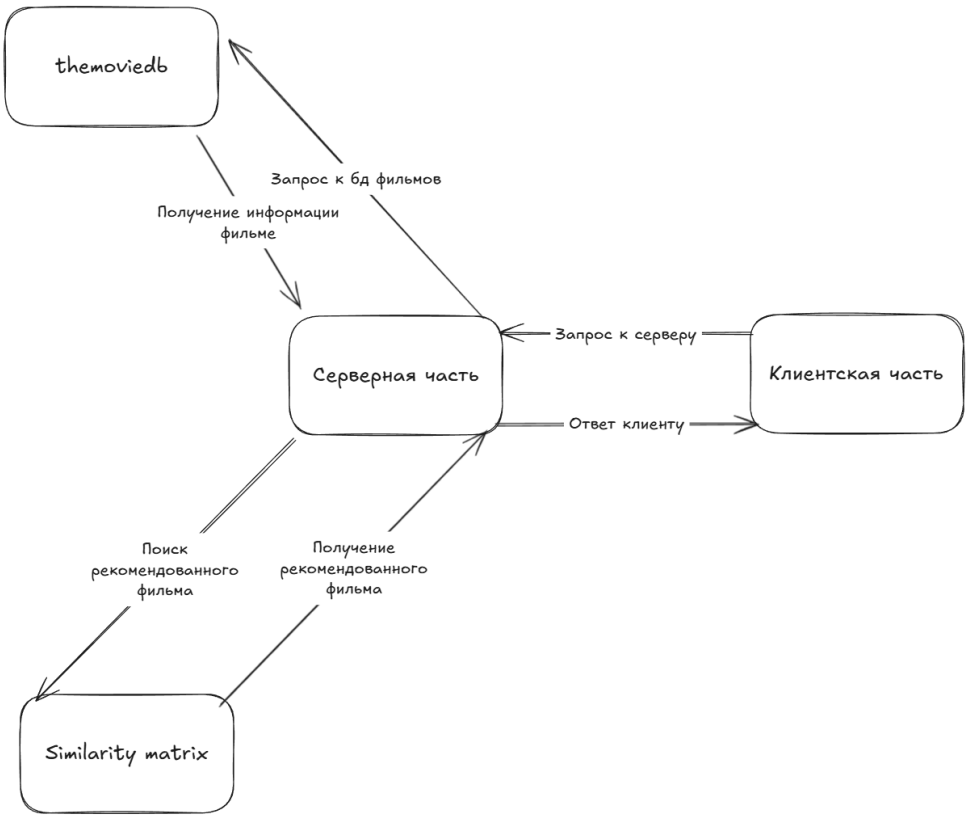

Рисунок 1 — Схема работы всего приложения.

Similarity matrix — Это заранее подготовленная «таблица схожести» всех фильмов друг на друга. Она построена один раз: каждый фильм превращён в набор слов (описание + жанры), посчитано, насколько фильмы похожи по этим словам (косинусное сходство), и всё сохранено в файл. Это сердце системы — именно по этой таблице и происходят все рекомендации.

Серверная часть — При старте загружает в память два файла: список фильмов и матрицу похожести. Когда приходит запрос от пользователя, сервер мгновенно:

- находит выбранный фильм,
- смотрит в матрице, какие фильмы на него больше всего похожи,

- берёт пятёрку лидеров,
- ходит в TheMovieDB, чтобы взять красивые постеры к этим фильмам,
- отдаёт клиенту готовый результат.

Клиентская часть — Это то, что видит пользователь в браузере. Простой и красивый интерфейс.

TheMovieDB — Внешняя база данных фильмов. Используется только для того, чтобы показывать актуальные и качественные постеры. Никакого влияния на сами рекомендации не оказывает — только визуальная часть.

Заключение

В рамках Практической работы была успешно решена задача выбора и проектирования рекомендательной системы фильмов.

Ключевым элементом системы стал классический и в то же время высокоэффективный алгоритм контентных рекомендаций, использующий векторизацию текстовых метаданных (описание + жанры) методом Bag-of-Words и косинусное сходство как меру близости объектов. Выбор именно этого алгоритма обусловлен его прозрачностью, полной интерпретируемостью результатов, возможностью мгновенной выдачи рекомендаций благодаря предвычисленной матрице сходства.

Для реализации проекта был тщательно подобран современный технологический стек, соответствующий лучшим практикам 2025 года:

- серверная часть — FastAPI, pandas, scikit-learn, pickle и TheMovieDB API;
- клиентская часть — React + TypeScript, Vite, shadcn/ui

Такое сочетание обеспечило высокую производительность, красивый внешний вид интерфейса и минимальное время отклика даже на датасетах в десятки тысяч объектов.

Была спроектирована и реализована чёткая клиент-серверная архитектура: тяжёлая вычислительная нагрузка выполняется один раз на этапе обучения, а в режиме онлайн-приложения все операции сводятся к чтению предвычисленной матрицы сходства и обогащению результата визуальными данными.

Таким образом, разработанное приложение представляет собой полностью рабочий, эстетически привлекательный и технически грамотный продукт, демонстрирующий весь цикл создания современной рекомендательной системы — от выбора и обоснования алгоритма до развёртывания полноценного веб-приложения с актуальным стеком технологий. Полученный результат может служить основой для дальнейшего развития: перехода к гибридным моделям, добавления персонализации и интеграции в реальные киноплатформы.

Список используемых источников

1. Документация Python. [Электронный ресурс]. Режим доступа: URL: https://www.python.org/doc/ (дата обращения 06.12.2026).
2. Документация Pandas. [Электронный ресурс]. Режим доступа: URL: https://pandas.pydata.org/docs/ (дата обращения 06.12.2026).
3. Документация Pickle. [Электронный ресурс]. Режим доступа: URL: https://docs.python.org/3/library/pickle.html (дата обращения 06.12.2026).
4. Документация FastApi. [Электронный ресурс]. Режим доступа: URL: https://fastapi.tiangolo.com/ (дата обращения 06.12.2026).
5. Документация requests. [Электронный ресурс]. Режим доступа: URL: https://pypi.org/project/requests/ (дата обращения 06.12.2026).
6. Документация themoviedb. [Электронный ресурс]. Режим доступа: URL: https://developer.themoviedb.org/reference/getting-started (дата обращения 06.12.2026).
7. Документация React. [Электронный ресурс]. Режим доступа: URL: https://react.dev/ (дата обращения 06.12.2026).
8. Документация TypeScript. [Электронный ресурс]. Режим доступа: URL: https://www.typescriptlang.org/ (дата обращения 06.12.2026).
9. Документация shadcn-ui. [Электронный ресурс]. Режим доступа: URL: https://ui.shadcn.com/ (дата обращения 06.12.2026).
10. Документация Vite. [Электронный ресурс]. Режим доступа: URL: https://vite.dev/ (дата обращения 06.12.2026).
11. Документация Axios. [Электронный ресурс]. Режим доступа: URL: https://axios-http.com/docs/intro (дата обращения 06.12.2026).